Case Study Engine: full documentation

This document explains end to end what the project is, how it works, how to run it, what it produced, and how it meets the judging criteria and the hackathon rules. It describes the client only as "an Indian cement manufacturer". The project library itself is confidential hackathon material (see section 13).

---

1. The problem

Companies that sell to enterprises win deals with proof: "we did this for a company like yours, and here is what changed." But most clients sign confidentiality agreements, so the best stories stay locked in email threads and project folders. Today a person spends days digging out results, guessing what is safe to say, and chasing the client for permission.

The hackathon (PanScience, 31 Oct 2026) gave each candidate the full record of one fictional 8-month project, **Project Drishti**: an AI safety-camera and gate-analytics deployment at a cement plant, January to September 2026. That's 42 files: contract clauses, scoping call, RFQ, proposal, signed requirements (BRD), purchase orders, change requests, tracker, incident report, acceptance certificate, three meeting transcripts, eight email threads, raw CSV exports, a code repository snapshot, a WhatsApp export, a LinkedIn post, images, the client's public profile and the vendor's draft sales notes.

The library is messy on purpose:

- targets were renegotiated and the sponsor's EHS head left mid-project;
- a feature was removed for privacy reasons;
- numbers were measured differently at different times, and results had several causes;
- the vendor's own sales team wrote claims the client had already rejected.

The task: build an AI agent that turns such a library into a case study a buyer believes and the client's finance and legal teams would sign off. It must produce six outputs, run as a chain of agents, refuse anything it can't trace, show its working, stop for human approval, and work on another project's library without being rewritten.

---

2. What I built, in one paragraph

A **chain of agents** in Python.

- **The LLM does the judgement:** reading the contract, choosing which KPI definitions are right, judging exposure risk, and writing for three readers. The prompts are in `engine/prompts/`.
- **Code does the counting and checking:** indexing files, computing every number, scanning for identifiers, verifying every sentence, and gating the release. These steps can't hallucinate.

Every sentence in every output carries a citation to a file and line, or to a calculation. A verifier refuses wrong numbers, drifted citations, identifiers and unmarked estimates. In live mode a fixer agent gets the refused lines back. A gate refuses to release anything until the people the contract names have approved the exact files, identified by their hash. A register records every project's status and history.

---

3. Architecture

```
 project library (any folder)
        |
 [1 Librarian]     code   evidence index: every file, every numbered line, image EXIF/GPS
        |
 [2 Rules]         LLM    reads the CONTRACT FIRST: publicity rules, required approvers, banned terms
+ safe replacements
        |
 [3 Results spec]  LLM    metric_spec.json: KPI definitions, baselines, windows, filters, headline
list (no arithmetic)
        |
 [4 Calculator]    code   calc_ledger: every number with its working; estimates flagged "assumed"
        |
 [5 Exposure]      code   regex + metadata scan (emails, IPs, PO numbers, URLs, GPS in photos, banned
terms)
                   + LLM  judged report: direct identifiers, combination risks, media, vendor claims
to avoid
        |
 [6 Writers]       LLM    story spine, results sheet, exposure report, 3-reader case study, approval
request
 [6 Media]         code   charts, diagram, labelled illustration, ROI calculator (from data; metadata
stripped)
        |
 [7 Verifier]      code   refuses: no citation, number not in cited source, citation unrelated to
claim,
        |                 identifier in outward text, estimate not marked ASSUMED, metadata in images
        |  refused --> [7b Fixer] LLM (live mode, up to 2 rounds) --> verifier again
        |  still refused --> no pack (and any older pack is deleted)
 [8 Gate]          code   assembles + hashes the pack, STOPS.
        |                 release: internal approval -> client Corporate Comms + Legal -> 10 business
days
        |                 -> no reject, no withdrawal -> pack unchanged since approval
 [Register]        code   runs/register.json|md: status + append-only history for every project
 [Viewer]          code   runs/<project>/viewer.html: dashboard + every output with hoverable
citations
```

Why split it this way: an LLM is good at reading messy documents and judging what matters. It is bad at arithmetic and at being consistently strict. So the LLM writes a *spec* ("K1 = compliant ÷ observed, pilot cameras, weeks 1–2"), and code does the maths. The LLM writes prose with citations, and code checks every citation.

---

4. The agents

#	Agent	Type	Output	File
1	Librarian	code	`evidence_index` (IDs D01–D42, numbered lines, image metadata)	`engine/agents/ingest.py`
2	Rules reader	LLM	`02_rules_and_terms.json`: 12 rules, approvers, 38 terms with replacements	`prompts/02_rules.md`

#	Agent	Type	Output	File
3	Results spec	LLM	`03_metric_spec.json`: 44 calcs + 6 headline KPIs	`prompts/03_results_spec.md`
4	Calculator	code	`calc_ledger` (value, working, filters, ASSUMED flag)	`agents/calc.py`
5	Exposure	code + LLM	`exposure_scan.md` + `03_exposure_report.md`	`agents/exposure.py`, `prompts/05_exposure.md`
6a	Timeline	LLM	`01_story_spine.md`	`prompts/06_timeline.md`
6b	Results sheet	LLM	`02_results_sheet.md`	`prompts/06_results_sheet.md`
6c	Writer	LLM	`04_case_study.md` (gatekeeper, user, buyer, champion)	`prompts/06_writer.md`
6d	Approval	LLM	`06_approval_request.md`	`prompts/06_approval.md`
6e	Media	code	4 PNGs + ROI calculator	`agents/media.py`
7	Verifier	code	`verifier_report.md`	`agents/verify.py`
7b	Fixer	LLM	corrected drafts (live mode)	`prompts/07_fix.md`
8	Gate	code	release OK / STOP with reasons	`agents/gate.py`
–	Register	code	`runs/register.json|md`	`agents/register.py`
–	Viewer	code	`viewer.html`	`agents/viewer.py`
–	LLM wrapper	code	live Gemini 3 call or replay of saved output	`agents/llm.py`

All LLM agents also receive `prompts/00_shared_rules.md`:

- cite everything, never compute, invent nothing;
- source priority when sources disagree: signed documents > raw data > client finance/legal > transcripts > vendor sales material;
- share credit, and don't hide missed targets.

Citation format

- `[D06:L45]` means line 45 of file D06; `[D06:L40-48]` is a range (file IDs are listed in `DATA.md`).
- `[C:k1_final]` means the calculation `k1_final` in the ledger.
- `[NO-CLAIM]` marks a line with no factual content. It is still checked for numbers and banned terms.

Verifier rules

1. Every claim line has at least one citation.
2. Cited files, line ranges and calc IDs exist.
3. Every number in the line appears in the cited source lines or the cited calculation (rounding allowed).
4. Every cited source line is **relevant**: it shares at least one content word or number with the claim, and is not blank. This catches citations that drifted to the wrong line.
5. A line citing an estimate must say **ASSUMED**, and ASSUMED must cite an estimate.
6. Outward text (the case study) contains no banned term.
7. Media files carry no metadata.

Dates, times, version tags, IDs (K1, CR-03) and clause numbers are not treated as quantities. Their lines still need a citation.

Gate rules

- Approvals are recorded per stage (`internal`, then `client`) and per role, bound to the pack's SHA-256 hash.
- Client release requires internal release first, plus **both** `client_corporate_communications` and `client_legal`. This follows contract clause 15.3: plant staff alone can't approve, so a WhatsApp yes from the plant head is refused.
- At least 10 business days between `--sent-on` and `--first-use` (clause 15.2e).
- `reject` stops release and prints the "on a no" steps; a withdrawal stops release (clause 15.4).
- Editing any file after approval changes the hash and voids the approvals.

---

5. Folder layout

```
submission/
├── README.md  DOCUMENTATION.md  DEMO_SCRIPT.md  PROCESS.md  DATA.md  .env.example
├── PROMPT_LOG.md  MANUAL_EDITS_LOG.md  PITCH.pptx  requirements.txt  docs_pdf/
├── engine/  run.py, agents/, prompts/
├── tests/test_engine.py           12 checks
├── demo/   make_bad_draft.py, reset_demo.py
├── scripts/ make_pitch.py, make_pdfs.py
├── library/                       the confidential project library + the brief (keep the repo
private)
└── runs/
    ├── register.json|md           all projects (local state)
    └── drishti/
        ├── agent_outputs/         LLM outputs (committed; replayed without an API key)
        ├── evidence_index.*  calc_ledger.*  exposure_scan.md  verifier_report.md  run_log.txt
(regenerated)
        ├── media/  pack/  viewer.html
(regenerated)
        └── approvals/             internal.json, client.json (created by approve)
```

---

6. How to run it

```
pip install -r requirements.txt
```

The library is in `library/Project_Drishti_Library/` (or pass `--library <folder>` for another project).

Action	Command	Result
Build	`python engine/run.py build --project drishti`	exit 0 = pack assembled, 2 = refused, 4 = no library
Approve	`python engine/run.py approve --project drishti --stage internal --role vendor_delivery_owner --name "Delivery Manager" --decision approve`	approval recorded against the hash

Action	Command	Result
Release	`python engine/run.py release --project drishti --stage client`	OK, or STOP with reasons (exit 3)
Say no	`... approve ... --decision reject --note "reason"`	register shows declined; "on a no" steps printed
Withdraw	`python engine/run.py withdraw --project drishti --stage client --role client_legal --date 2027-01-10`	release blocked from then on
Review	open `runs/drishti/viewer.html`	dashboard + all outputs
Test	`python -m pytest -q tests`	12 passed
Refusal demo	`python demo/make_bad_draft.py` then `build --project drishti_bad`	refused, 6 problems
Reset	`python demo/reset_demo.py`	clean demo state

Live LLM mode (Google Gemini 3): copy `.env.example` to `.env` and set `GEMINI_API_KEY` (optionally `GEMINI_MODEL`; default `gemini-3-pro-preview`, falling back to `gemini-3-flash-preview`). Without a key the saved agent outputs are replayed and marked RECORDED.

---

7. The review screen (viewer.html)

- **Overview dashboard:**
  - headline KPI cards (baseline → result, target, met / missed, what else contributed);
  - the agent chain with each agent's status from the run log (code / LLM, LIVE / RECORDED);
  - the approval gate (verifier, internal and client steps);
  - how the run answers each judging criterion, with buttons to the evidence;
  - the project register.
- **One tab per output.** Blue chips show the exact source lines; green chips show the calculation working.
- **Media tab** with every chart and the interactive ROI calculator.
- **Verifier and run-log tabs.**
- Light/dark theme toggle. It is one self-contained file: images are embedded, so it opens offline by double-click.

---

8. The six outputs

1. **Story spine:** 37 dated events from the first email (28 Jan) to the public post (30 Sep), each with who decided, why it matters and its source. Human turning points are marked, followed by a promised / signed / delivered table.
2. **Results sheet + calculation ledger:** each KPI with definition, baseline and window, result, target, met or not, and what else contributed. Also covers post-scale results, gate time split by cause, and money (measured vs ASSUMED vs not claimed), plus "where sources disagree" and "numbers we refuse to use".
3. **Exposure report:** 16 direct identifiers with their safe replacement and contract rule, combination risks, media review, and vendor-side claims not to repeat.

4. **Case study** (traced copy + clean copy for the client): headline, a fact sheet for the gatekeeper, a day before and after for the user, the business view for the buyer, "what we got wrong and fixed", and a note the champion can forward.
5. **Media pack:** PPE by camera cohort, gate turnaround with causes marked, generic on-prem architecture, a labelled alert illustration, an ROI calculator (example inputs only).
6. **Approval request:** cover email to Corporate Comms cc Legal, reply by **Mon 16 Nov 2026** (10 business days from 2 Nov). It includes per-asset options (named / anonymised / internal only / no), what was changed and why, every number with its source, signature lines, and an offer of the charts for the client's own council deck.

---

9. What the engine found

Results it stands behind. All are recomputed from raw data, and the pilot rows match the signed acceptance certificate exactly.

KPI	Baseline	Result	Target	Status
PPE compliance (helmet + vest, on camera)	68.9%	91.1% (pilot) → 94.4% (pilot areas, Sep)	≥ 90%	Met
Restricted-zone breaches per 1,000 camera-hours	5.58	3.00 (−46%) → 1.26 (−77%, Sep)	−50%	Missed in pilot; met after the client's guards
Alert acknowledgement	9.3 min (week 1)	4.9 min → about 2.1 min (Sep)	≤ 3 min	Missed in pilot; met after phone escalation
False alarms	20.1% before retrain	9.7% → 6.2% (Sep)	< 10%	Met
Truck turnaround (same cameras throughout)	88.4 min	67.6 min (−24%)	−15%	Met: about 5.6 min after the client's new weighbridge, about 15.2 min after dispatch slots

Claims it refuses:

- "74% → 97%": mixes the announced-walk baseline with one zone's best week.
- "Zero LTIs" as a result: seven months proves nothing, and the client refused to claim it.
- "₹2.3 Cr saved": counts benchmark and avoided-injury savings that Finance rejected.
- "9 → 2 min" credited to the software alone.
- Rising near-miss reports: the definition changed mid-series.

Credit shared: the safety week, daily toolbox talks, contractor discipline, physical guards, control-room escalation, a second weighbridge, and dispatch slots run by logistics.

Exposure caught:

- GPS coordinates and the plant name embedded in the snapshot file;
- signage and the company name in the images;
- internal IPs and RTSP URLs in the repo;
- the plant's capacity, which is in the annual report;
- the Head of EHS's public post naming the vendor (a combination risk);
- an award nomination that named the client without approval.

Data quality:

- The survey's last four comments duplicate the first four word for word, so they are not quoted.
- "15 h vs 41 h": one figure is the outage's detection delay, the other its length.
- Tracker and changelog dates differ: one marks the merge, the other the deploy.
- The signed results certificate's 4.9 min was reproduced as an alert-weighted figure.

---

10. Tests (12, all passing)

#	Check	Type
1	Calculator reproduces every signed pilot KPI exactly	correctness
2	"74% → 97%" refused even though it cites real calcs	**refuse a claim**
3	Uncited sentence and citation to a missing file refused	refuse
4	Plant name refused in outward text, allowed internally	exposure
5	ASSUMED must cite an estimate	honesty
6	Finds GPS in the snapshot and RTSP URLs in the repo	exposure
7	No approval → release stops	**stop for a person**
8	Plant head's WhatsApp yes is not enough	stop
9	Edit after approval / short notice / withdrawal each stop release	stop
10	Same code runs a different (software) project with a new spec	generality
11	Drifted citation and unmarked estimate refused	refuse
12	A client "no" blocks release; register keeps history	process

---

11. How it answers the four judging criteria

Criterion (25 each)	What the judges ask	Evidence
Agentic workflow	Agents properly divided? Read, verify, calculate, work together?	8 agents + fixer, each with one job; LLM judges, code computes and checks; hand-offs via files; verifier → fixer loop; human gate; runs on another library (test 10); run log and dashboard show every agent
Business understanding	Promised vs delivered? Numbers accurate? Other factors acknowledged?	Promised / signed / delivered table; signed KPIs reproduced exactly; missed targets shown with remediation; a "what else contributed" column; refused numbers explained; money only where Finance measured it
Buyer psychology	Different writing for buyer, user, gatekeeper? Client comfortable approving?	Same facts told three ways + champion note; the user's turning points are human (snapshot ended arguments, gang leaders self-check); the approval request goes to the right people, sets no external deadline, offers options per asset, credits their teams and offers them the charts
Process design	Approval, rejection, records, many projects?	PROCESS.md; approve / reject / withdraw commands; hash-bound approvals; notice period; register with history; one code path for every project

---

12. Hackathon rules: how each is met

Rule	How
1. Work alone	Built by one person with AI tools. Sharing with a teammate must wait until after judging (see section 13)
2. Any AI tools	AI coding tools were used to build it and are logged in PROMPT_LOG.md; the engine's own LLM provider is Gemini 3
3. Only the library for facts	Every claim cites a library line or a calc on library data; the verifier refuses anything else
4. Invent nothing	No invented results, quotes or people; quotes are only used where the transcript has them; the approval email is signed by a role from the library
5. Mark estimates ASSUMED	Enforced by the verifier (rules 5); the September detention figure, annualised saving and payback are all marked
6–7. Confidential, don't upload or share	Outward text is anonymised; the repository holding the library is private, and credentials are rotated after use
8. Nothing leaves the room	Nothing published or sent; the gate blocks release by design
9. Complete and verified	Verifier 0 problems, 12 tests pass, every screen and command checked

13. Limitations and sharing

Limitations:

- In this run no API key was available, so the LLM agents' outputs were written with these prompts in an AI coding tool (see PROMPT_LOG.md) and are replayed. With `GEMINI_API_KEY` set they run live on Gemini 3. The code agents always run live.
- The verifier checks numbers, citation existence and relevance, identifiers and ASSUMED marking. It does not prove a sentence's full meaning or check date values; the internal human review covers that.
- The media module is built for manufacturing. Other industries need their own (the writer prompt asks it to choose by industry).

Sharing:

- The hackathon rules say to work alone, to keep the library confidential and not share it, and that nothing leaves the room during the hackathon.
- Share the repo with a teammate only **after** judging, and only with the organisers' agreement.
- The repository is private and includes the library; keep it private and rotate credentials after use (see DATA.md).